

NLP Exam Revision Notes

Byte-Pair Encoding • Edit Distance • Lesk Algorithm • TF-IDF (LSI & FSA) • N-gram Models • Regular Expressions • POS Tagging (HMM/CRF) • Word Sense Disambiguation

1. Byte-Pair Encoding (BPE)

1.1 What it is

Byte-Pair Encoding is a subword tokenization algorithm. It was originally a data-compression technique (Gage, 1994) that repeatedly replaces the most frequent pair of adjacent bytes with a new byte. Sennrich et al. (2016) adapted it to NLP to build a subword vocabulary for neural machine translation and other models (GPT, RoBERTa use byte-level BPE).

1.2 Why subword tokenization

- Pure word-level tokenization cannot handle out-of-vocabulary (OOV) words.
- Pure character-level tokenization avoids OOV but produces very long sequences and loses word-level meaning.
- BPE sits in between: frequent words remain whole tokens, rare/unseen words are broken into meaningful subword units (e.g. "unhappiness" → "un", "happi", "ness").
- Keeps vocabulary size fixed and controllable while keeping sequences reasonably short.

1.3 Training Algorithm

1. Initialize the vocabulary with every individual character appearing in the training corpus (plus a special end-of-word symbol, e.g. "_" or "</w>").
2. Represent every word in the corpus as a sequence of these characters, keeping track of word frequency.
3. Count the frequency of every pair of adjacent symbols across the whole corpus.
4. Find the most frequent pair (A, B) and merge it into a single new symbol "AB"; add "AB" to the vocabulary.
5. Replace every occurrence of the pair (A, B) in the corpus with the new merged symbol "AB".
6. Repeat steps 3–5 for a fixed number of merge operations k (a hyperparameter), or until the desired vocabulary size is reached.

The output of training is an ordered list of merge rules. This ordering matters at encoding time — merges learned earlier are applied first.

1.4 Worked Example

Toy corpus (word: frequency):

Word	Frequency	Initial symbol sequence
low	5	l o w _
lowest	2	l o w e s t _
newer	6	n e w e r _
wider	3	w i d e r _

Iteration 1: Count all adjacent pairs across the weighted corpus. The pair (e, r) appears in "newer" (6) and "wider" (3) = 9 times — the most frequent pair. Merge 'e r' → 'er'.

newer → n e w er _ wider → w i d er _

Iteration 2: Now (er, _) appears 6 + 3 = 9 times — most frequent. Merge 'er _' → 'er_'.

newer → n e w er_ wider → w i d er_

Iteration 3: Pair (l, o) appears in "low" (5) and "lowest" (2) = 7 times. Merge 'l o' → 'lo', and continue merging ('lo w' → 'low', etc.) for as many merges as the budget allows.

This continues until the merge budget k is exhausted. The final vocabulary contains characters plus all merged subwords ("er_", "low", ...).

1.5 Encoding / Decoding New Text

- Encoding: split the new word into characters, then apply the learned merge rules in the exact order they were learned, merging whichever adjacent pair matches next, until no more merges apply.
- Decoding: simply concatenate the subword tokens and strip the end-of-word marker.

1.6 Key Points for Exams

- BPE is a greedy, bottom-up, frequency-based merge algorithm.
 - Number of merges k directly controls final vocabulary size.
 - Deterministic given the corpus and k — always merges the single most frequent pair at each step.
 - Variants: byte-level BPE (operates on raw UTF-8 bytes so any Unicode string is representable with no <UNK>), BPE-dropout (randomly skips merges during training for regularization/robustness).
 - Related subword methods: WordPiece (uses likelihood increase instead of raw frequency), SentencePiece, Unigram LM tokenizer.
-

2. Edit Distance Algorithm and Table Construction

2.1 Definition

The (Levenshtein) edit distance between two strings is the minimum number of single-character edit operations — insertion, deletion, substitution — required to transform one string into the other. It is solved by dynamic programming.

2.2 Recurrence Relation

Let source string X have length n and target string Y have length m . $D(i, j)$ = edit distance between the first i characters of X and the first j characters of Y .

Base cases: $D(i, 0) = i$ $D(0, j) = j$

$$D(i, j) = \min [D(i-1, j) + del_cost, \quad D(i, j-1) + ins_cost, \quad D(i-1, j-1) + sub_cost(x_i, y_j)]$$

where $sub_cost(x_i, y_j) = 0$ if $x_i = y_j$, else 1 (Levenshtein distance), or 2 in the classical Levenshtein-with-substitution-cost-2 variant used in some textbooks (equivalent to a delete + insert).

2.3 Algorithm (Wagner–Fischer)

1. Create a table D of size $(n+1) \times (m+1)$.

2. Initialize row 0 as 0,1,2,...,m and column 0 as 0,1,2,...,n.
3. Fill the table row by row (or column by column) using the recurrence above.
4. The value $D(n, m)$ in the bottom-right cell is the edit distance.
5. To recover the actual sequence of operations, keep a backpointer at each cell recording which of the three options was chosen, then trace back from $D(n,m)$ to $D(0,0)$.

2.4 Worked Example: "intention" → "execution"

Using insertion cost = 1, deletion cost = 1, substitution cost = 2 (classic textbook version):

	#	e	x	e	c	u	t	i	o	n
#	0	1	2	3	4	5	6	7	8	9
i	1	2	3	4	5	6	7	6	7	8
n	2	3	4	5	6	7	8	7	8	7
t	3	4	5	6	7	8	7	8	9	8
e	4	3	4	5	6	7	8	9	10	9
n	5	4	5	6	7	8	9	10	11	10
t	6	5	6	7	8	9	8	9	10	11
i	7	6	7	8	9	10	9	8	9	10
o	8	7	8	9	10	11	10	9	8	9
n	9	8	9	10	11	12	11	10	9	8

Final answer: $D(9,9) = 8$, the minimum edit distance between "intention" and "execution" (with substitution cost 2).

Filling method for any cell: look at the cell above (+1, deletion), the cell to the left (+1, insertion), and the diagonal cell (+0 if the two characters match, +sub_cost otherwise); take the minimum of the three.

2.5 Complexity

- Time complexity: $O(n \times m)$ — every cell computed once in constant time.
- Space complexity: $O(n \times m)$ for the full table (needed if backtrace/alignment is required); can be reduced to $O(\min(n,m))$ if only the distance value is needed (keep only two rows).

2.6 Variants

- Damerau–Levenshtein distance: additionally allows transposition of two adjacent characters as a single operation (handles common typos like "teh" → "the").
- Weighted edit distance: different costs for different operations/characters (e.g. keyboard-adjacency-aware costs for spelling correction).
- Applications: spell checking, DNA sequence alignment, plagiarism detection, minimum edit distance-based sentence similarity.

3. Lesk Algorithm for Word Sense Disambiguation

3.1 Idea

The Lesk algorithm is a knowledge-based / dictionary-based approach to WSD. Its core assumption: words used together in a sentence tend to share a common topic, so the correct sense of an ambiguous word is the one whose dictionary definition (gloss) has the greatest word overlap with the glosses/context of the surrounding words.

3.2 Original Lesk Algorithm

1. For each ambiguous word in the sentence, retrieve all candidate senses and their glosses from a dictionary (e.g. WordNet).
2. For every combination of senses of the words in the context window, compute the pairwise overlap (number of shared words) between the glosses.
3. Choose the combination of senses that maximizes total overlap across all word pairs.

Problem: this is combinatorially expensive — the number of sense combinations grows exponentially with the number of ambiguous words in the context.

3.3 Simplified Lesk Algorithm (commonly used / commonly examined)

Disambiguates one target word at a time against the words in its surrounding context, instead of jointly optimizing all words.

1. Identify the target word w and collect the context words C (a window of surrounding words in the sentence).
2. For each sense s of w , retrieve its gloss (definition) and, optionally, example sentences from the dictionary/WordNet.
3. Compute the overlap score = number of common (non-stopword) tokens between the gloss of sense s and the set of context words C .
4. Select the sense s^* with the maximum overlap score: $s^* = \operatorname{argmax}_s |\text{gloss}(s) \cap \text{context}(w)|$
5. If there is a tie, fall back to the most frequent sense (first sense listed in WordNet).

3.4 Worked Example

Sentence: “The river bank was flooded.” Target word: “bank”.

Sense of ‘bank’	Gloss	Overlap with context {river, flooded}
Financial institution	an establishment for keeping money, valuables, safe deposit	0
Sloping land beside a river	sloping land, especially the slope beside a body of water such as a river	1 (river)

The “sloping land beside a river” sense wins because it overlaps with “river” in the context, correctly disambiguating “bank” as the geographical sense.

3.5 Limitations

- Very sensitive to exact wording of dictionary glosses — short glosses give sparse, unreliable overlap counts.
- Performance depends heavily on dictionary quality and coverage.
- Does not use any statistical/corpus evidence, only lexicographic definitions.

3.6 Extensions

- Corpus/Extended Lesk: also includes glosses of related synsets (hypernyms, hyponyms, meronyms) from WordNet to enlarge the overlap set.
- Weight overlapping words by IDF or by length of the matching n-gram (not just raw word count) to reduce noise from common words.

4. TF-IDF with Latent Semantic Indexing (LSI)

4.1 TF-IDF Recap

$tf(t, d) = \text{number of times term } t \text{ occurs in document } d$ (or log-normalized: $1 + \log(\text{count})$ if count > 0, else 0)

$idf(t) = \log(N / df(t))$ where $N = \text{total number of documents}$, $df(t) = \text{number of documents containing } t$

$tf-idf(t, d) = tf(t, d) \times idf(t)$

Intuition: a term is important to a document if it occurs frequently in that document (high tf) but rarely across the whole collection (high idf, i.e. it is discriminative).

4.2 Term-Document Matrix

Build a matrix A of size (terms $\times$ documents) where each entry $A[i][j]$ = tf-idf weight of term i in document j . This matrix is typically very high-dimensional and sparse, and it treats each term as an independent axis — it cannot capture synonymy (different words, same meaning) or polysemy (same word, different meanings).

4.3 Latent Semantic Indexing — Applying SVD

LSI applies Singular Value Decomposition (SVD) to the tf-idf weighted term-document matrix to uncover latent (hidden) semantic structure and reduce dimensionality.

$$A (t \times d) = U (t \times r) \cdot \Sigma (r \times r) \cdot V^T (r \times d)$$

- U : term-concept matrix — how each term relates to each latent concept.
- Σ : diagonal matrix of singular values, ranked by importance/variance explained.
- V^T : concept-document matrix — how each document relates to each latent concept.
- r = rank of A .

4.4 Truncated SVD (dimensionality reduction)

Keep only the top k singular values ($k \ll r$) and the corresponding columns of U and V , discarding the rest as noise:

$$A_k = U_k \cdot \Sigma_k \cdot V_k^T \quad (\text{best rank-}k \text{ approximation of } A, \text{ by the Eckart-Young theorem})$$

This projects both terms and documents into a shared k -dimensional “latent semantic space” where terms that co-occur across many documents (synonyms) end up close together, even if they never co-occur in the same document.

4.5 Full LSI Procedure

1. Build the term-document matrix from the corpus (raw counts).
2. Weight it with tf-idf.
3. Compute the SVD of the weighted matrix.
4. Choose k (number of latent concepts to retain) and truncate U , Σ , V .
5. Represent every document as its row in $\Sigma_k V_k^T$ (its k -dimensional latent vector).
6. To handle a new query, fold it in: $q_k = q^T \cdot U_k \cdot \Sigma_k^{-1}$, then compare q_k to document vectors using cosine similarity.

4.6 Why LSI Helps

- Synonymy: different words with related meaning get mapped near each other in latent space, improving recall (e.g., a query for “car” can match a document that only says “automobile”).
- Polysemy: is partially handled since a word’s vector becomes an average over its different usages/contexts.
- Dimensionality reduction removes noise and sparsity, and reveals underlying “topics”/concepts.

- Used in information retrieval, document clustering, document similarity ranking.
-

5. TF-IDF with Finite State Automata (FSA)

5.1 Role of FSA in the TF-IDF Pipeline

Before term frequencies can be counted, raw text must be scanned and broken into valid tokens (words, numbers, punctuation-delimited units). This scanning/tokenization step can be formally modeled as recognition by a finite state automaton: a regular expression is written to define what counts as a valid “term”, and that regular expression is compiled into an automaton that scans the document one character at a time.

5.2 Pipeline

1. Define the token pattern as a regular expression, e.g. a word = one or more letters, optionally with internal apostrophes/hyphens: $[A-Za-z]+([\ '-][A-Za-z]+)^*$
2. Convert the regular expression to an NFA (Thompson’s construction), then determinize it to a DFA (subset construction), and minimize it if desired.
3. Run the DFA over the input document, character by character. Every time the automaton reaches an accepting state and the next character does not continue a valid transition (maximal munch), the accepted substring is emitted as one token/term.
4. Maintain a running count for each distinct term recognized — this produces the raw term-frequency counts $tf(t, d)$ needed for tf-idf.
5. Repeat across all documents in the collection to additionally obtain document frequency $df(t)$ (number of documents in which the automaton accepts the term at least once), from which $idf(t) = \log(N/df(t))$ is computed.
6. Combine: $tf-idf(t, d) = tf(t, d) \times idf(t)$, exactly as in the standard formula, but here the tf counts are guaranteed to only include strings the automaton recognizes as well-formed terms (so punctuation, stray symbols, etc. never pollute the counts).

5.3 Why Use an FSA Formulation

- Efficiency: DFA-based scanning processes text in a single left-to-right pass, $O(\text{length of text})$, with $O(1)$ work per character — ideal for tokenizing very large corpora before tf-idf computation.
 - Precision/consistency: the automaton gives an exact, unambiguous, reproducible definition of what is/is not a valid term (removing ad-hoc string-splitting bugs).
 - Extensibility: more complex term patterns (e-mail addresses, numbers with decimals, hashtags, multi-word named entities) can be captured by extending the regular expression / automaton rather than hand-written string code.
 - Because DFAs can be combined (union, intersection) they can also encode stop-word filtering or normalization rules directly into the scanning stage before tf-idf statistics are accumulated.
-

6. N-gram Models, Sentence Probability & Perplexity

6.1 Chain Rule of Probability

The probability of a sentence $W = w_1 w_2 \dots w_n$ can be decomposed exactly using the chain rule:

$$P(w_1, w_2, \dots, w_n) = P(w_1) \times P(w_2/w_1) \times P(w_3/w_1, w_2) \times \dots \times P(w_n/w_1, \dots, w_{n-1})$$

This exact decomposition is intractable to estimate directly because the conditioning context grows without bound. N-gram models make a Markov (limited-history) assumption to make this tractable.

6.2 Markov Assumption / N-gram Approximation

An N-gram model assumes the probability of a word depends only on the previous N-1 words:

$$P(w_i | w_1, \dots, w_{(i-1)}) \approx P(w_i | w_{(i-N+1)}, \dots, w_{(i-1)})$$

- Unigram (N=1): $P(w_i)$ — no context.
- Bigram (N=2): $P(w_i | w_{(i-1)})$.
- Trigram (N=3): $P(w_i | w_{(i-2)}, w_{(i-1)})$.

6.3 Maximum Likelihood Estimation

$$\text{Bigram: } P(w_i | w_{(i-1)}) = \text{Count}(w_{(i-1)}, w_i) / \text{Count}(w_{(i-1)})$$

$$\text{Trigram: } P(w_i | w_{(i-2)}, w_{(i-1)}) = \text{Count}(w_{(i-2)}, w_{(i-1)}, w_i) / \text{Count}(w_{(i-2)}, w_{(i-1)})$$

Sentences are usually padded with a start symbol $\langle s \rangle$ and an end symbol $\langle /s \rangle$ so the first and last words also have well-defined contexts.

6.4 Sentence Probability — Worked Example (Bigram)

For sentence “ $\langle s \rangle$ I am happy $\langle /s \rangle$ ”:

$$P(\text{sentence}) = P(I | \langle s \rangle) \times P(\text{am} | I) \times P(\text{happy} | \text{am}) \times P(\langle /s \rangle | \text{happy})$$

Each factor is looked up/estimated from bigram counts in the training corpus, then all factors are multiplied together to get the overall sentence probability.

6.5 Smoothing (needed because MLE assigns 0 probability to unseen n-grams)

- Laplace / Add-one smoothing: $P(w_i | w_{(i-1)}) = [\text{Count}(w_{(i-1)}, w_i) + 1] / [\text{Count}(w_{(i-1)}) + V]$ where V = vocabulary size.
- Add-k smoothing: generalizes add-one with a tunable $k < 1$.
- Good-Turing smoothing: re-estimates the probability mass of unseen events using the counts of things seen exactly once.
- Kneser-Ney smoothing: state-of-the-art n-gram smoothing; uses absolute discounting plus a continuation probability that favors words that appear in many different contexts.
- Backoff / interpolation: fall back from higher-order n-grams to lower-order ones when the higher-order count is zero or unreliable.

6.6 Perplexity

Perplexity is the standard intrinsic evaluation metric for a language model: it measures how well the model predicts a held-out test set. Lower perplexity = better (less “surprised”) model.

$$PP(W) = P(w_1, w_2, \dots, w_N)^{(-1/N)}$$

Equivalently, using the chain rule expansion (general n-gram form):

$$PP(W) = [\prod_{(i=1 \text{ to } N)} 1 / P(w_i | w_{(i-N+1)}, \dots, w_{(i-1)})]^{(1/N)}$$

For a bigram model specifically:

$$PP(W) = [\prod_{(i=1 \text{ to } N)} 1 / P(w_i | w_{(i-1)})]^{(1/N)}$$

In log form (used in practice to avoid numerical underflow):

$$PP(W) = \exp(- (1/N) \times \sum_{i=1 \text{ to } N} \log P(w_i / \text{context}))$$

Perplexity can also be understood as 2 raised to the cross-entropy of the model on the test set: $PP(W) = 2^{H(W)}$, where $H(W) = -(1/N) \sum \log_2 P(w_i | \text{context})$.

6.7 Worked Perplexity Example

Suppose a bigram model assigns probabilities to a 4-word test sentence ($N = 4$, including $\langle /s \rangle$): $P(w_1)=0.5$, $P(w_2|w_1)=0.4$, $P(w_3|w_2)=0.2$, $P(w_4|w_3)=0.5$.

$$P(W) = 0.5 \times 0.4 \times 0.2 \times 0.5 = 0.02$$

$$PP(W) = (0.02)^{-(1/4)} = (1/0.02)^{(1/4)} = 50^{0.25} \approx 2.66$$

A perplexity of ~ 2.66 means the model is, on average, as “confused” as if it had to choose uniformly among about 2.66 equally likely words at each step.

6.8 Key Points

- Perplexity is only meaningful when comparing models over the same test set and same vocabulary.
- Perplexity is the inverse of the (geometric-mean) probability the model assigns to the test set, normalized by sentence length.
- Unknown words are handled via a special $\langle \text{UNK} \rangle$ token, whose probability is estimated during training (e.g., by replacing rare/singleton words with $\langle \text{UNK} \rangle$).

7. Regular Expressions and Their Representations

7.1 What is a Regular Expression

A regular expression (regex) is a formal, algebraic notation for specifying a set of strings — a regular language. It is built from a small set of operators applied to literal characters.

7.2 Core Operators

Operator	Meaning	Example
Concatenation	ab means a followed by b	cat matches “cat”
Union / Alternation	a b means a or b	cat dog
Kleene star *	zero or more repetitions	ba* matches b, ba, baa, ...
Kleene plus +	one or more repetitions	ba+ matches ba, baa, ... (not b)
Optional ?	zero or one occurrence	colou?r matches color/colour
Character class []	any one character from the set	[abc] matches a, b, or c
Range	range inside a class	[a-z], [0-9]
Negation [^]	any character NOT in the set	[^0-9] matches any non-digit
Wildcard .	any single character	b.g matches big, bag, bug
Anchors ^ \$	start / end of line	^The, end\$

7.3 Equivalent Representations of a Regular Language

A key theoretical result (Kleene’s theorem) is that the following four formalisms are all exactly equivalent in expressive power — each defines precisely the class of regular languages:

- Regular expressions (algebraic notation).

- Non-deterministic Finite Automata (NFA) — may have multiple/zero transitions for a symbol from a state, and ϵ (epsilon) transitions.
- Deterministic Finite Automata (DFA) — exactly one transition per symbol per state, no ϵ transitions.
- Regular grammars (right-linear or left-linear context-free grammars).

7.4 Regex $\rightarrow$ NFA (Thompson's Construction)

Every regex operator has a direct NFA-fragment building rule, and fragments are combined recursively following operator precedence:

- A single symbol a becomes a 2-state fragment with one transition on a .
- Concatenation RS : connect the accept state of R 's fragment to the start state of S 's fragment with an ϵ -transition.
- Union $R|S$: new start state with ϵ -transitions to both R and S start states; both accept states ϵ -transition to a new shared accept state.
- Kleene star R^* : new start/accept states; ϵ -transition from new start to R 's start (and directly to new accept, for zero occurrences); ϵ -transition from R 's accept back to R 's start (repetition) and to the new accept state.

7.5 NFA $\rightarrow$ DFA (Subset / Powerset Construction)

1. Compute the ϵ -closure of the NFA start state — this becomes the DFA start state (a set of NFA states).
2. For the current DFA state (a set of NFA states) and each input symbol, compute the set of NFA states reachable, then take its ϵ -closure — this set becomes a new DFA state (if not already created).
3. Repeat until no new DFA states are produced.
4. Any DFA state (set) that contains an NFA accepting state is marked as a DFA accepting state.

The resulting DFA may optionally be minimized (merge equivalent states) using Hopcroft's or Moore's algorithm to obtain the smallest possible DFA.

7.6 Why This Matters in NLP

- Tokenizers, lexical analyzers, and morphological analyzers are implemented as DFAs compiled from regular expressions describing valid tokens/word-forms.
- Finite-state transducers (FSTs) extend FSAs with an output tape and are used for morphological analysis/generation (mapping surface forms $\leftrightarrow$ lexical/tag forms).
- Regex-driven scanning is linear time, $O(n)$ in input length, because DFA execution does one table lookup per character.

8. Part-of-Speech (POS) Tagging — General, HMM, CRF

8.1 What is POS Tagging

POS tagging is the task of assigning a grammatical category (noun, verb, adjective, determiner, ...) to every word/token in a sentence, given its context. Standard tagsets include the Penn Treebank tagset (~45 tags).

Ambiguity is the central challenge: many words can take multiple tags depending on context, e.g. "book" can be a NOUN ("read a book") or a VERB ("book a flight").

8.2 Approaches Overview

- Rule-based tagging: hand-written disambiguation rules (e.g. the Brill tagger uses an initial most-frequent-tag assignment followed by learned transformation rules).
- Stochastic / probabilistic tagging: choose the tag sequence that maximizes a probability model — classic example: Hidden Markov Model (HMM).
- Discriminative sequence models: Conditional Random Fields (CRF), Maximum Entropy Markov Models (MEMM).
- Neural approaches: BiLSTM/BiLSTM-CRF, Transformer-based taggers (beyond classical syllabus scope but worth mentioning).

8.3 HMM POS Tagging

An HMM models the tag sequence as a Markov chain of hidden states (the POS tags) that emit the observed words. It is a generative model: it models the joint probability $P(\text{words, tags})$ and finds the tag sequence maximizing it.

$$\hat{t}(1:n) = \operatorname{argmax}(t1:n) P(t1:n / w1:n) = \operatorname{argmax}(t1:n) P(w1:n / t1:n) \times P(t1:n)$$

Applying two independence (Markov) assumptions simplifies this to a product of two kinds of probabilities:

- Emission (likelihood) probability: $P(w_i | t_i)$ — probability of a word given its own tag only.
- Transition probability: $P(t_i | t_{i-1})$ — probability of a tag given only the immediately preceding tag (bigram HMM).

$$\hat{t}(1:n) \approx \operatorname{argmax}(t1:n) \prod_{i=1 \text{ to } n} P(w_i / t_i) \times P(t_i / t_{i-1})$$

Both $P(w_i | t_i)$ and $P(t_i | t_{i-1})$ are estimated via MLE (relative frequency counting) from a tagged training corpus, with smoothing for unseen words/transitions.

8.3.1 Viterbi Decoding

Finding the argmax tag sequence by brute force is exponential in sentence length; the Viterbi algorithm solves it in polynomial time via dynamic programming.

1. Define $v_j(i)$ = the probability of the best tag path ending in tag j at position i .
2. Initialization: $v_j(1) = P(t_j | <s>) \times P(w_1 | t_j)$ for every tag j .
3. Recursion: $v_j(i) = \max$ over all previous tags k of $[v_k(i-1) \times P(t_j | t_k) \times P(w_i | t_j)]$, and store a backpointer to the best k .
4. Termination: the best final score is $\max$ over k of $v_k(n) \times P(</s> | t_k)$.
5. Backtrace through the stored backpointers from the best final state to recover the full optimal tag sequence.

Complexity: $O(n \times T^2)$, where n = sentence length and T = number of possible tags (since at each position every tag must consider every previous tag).

8.4 CRF (Conditional Random Field) POS Tagging

A linear-chain CRF is a discriminative model: instead of modeling $P(\text{words, tags})$ jointly and using Bayes' rule, it directly models the conditional probability of the tag sequence given the word sequence, $P(\text{tags} | \text{words})$, avoiding the need to model the distribution of the words themselves.

$$P(t1:n / w1:n) = (1/Z(w)) \times \exp[\sum_{i=1 \text{ to } n} \sum_k \lambda_k \cdot f_k(t(i-1), t_i, w1:n, i)]$$

- $f_k(\dots)$: arbitrary feature functions — can look at the current word, surrounding words, prefixes/suffixes, capitalization, the previous tag, word shape, etc. — not limited to just the current word and previous tag as in HMMs.
- λ_k : learned feature weights (trained discriminatively, e.g. via gradient-based optimization of conditional log-likelihood).
- $Z(w)$: the normalization constant (partition function) summing over all possible tag sequences for the given word sequence, ensuring probabilities sum to 1.

Because CRFs normalize globally over the whole sequence (rather than locally at each position, as MEMMs do), they avoid the “label bias problem”. Decoding (finding the best tag sequence) is again performed with a Viterbi-style dynamic program adapted to the CRF’s feature-based scores.

8.5 HMM vs CRF — Comparison

Aspect	HMM	CRF
Model type	Generative: models $P(w,t)$	Discriminative: models $P(t w)$ directly
Features	Only current word + previous tag (fixed structure)	Arbitrary, overlapping features of the whole input
Independence assumption	Strong (word depends only on own tag)	None required — can use rich, overlapping features
Training	MLE / relative frequency counting	Discriminative training (gradient-based, maximize conditional likelihood)
Typical accuracy	Good, but lower than CRF	Generally higher, given good features
Decoding	Viterbi algorithm	Viterbi-style algorithm on feature scores

9. Word Sense Disambiguation (General)

9.1 Definition

Word Sense Disambiguation (WSD) is the task of determining which sense (meaning) of an ambiguous (polysemous) word is being used in a given context, out of a fixed inventory of possible senses (typically taken from a lexical resource such as WordNet).

9.2 Why It Is Hard

- Many common words are highly polysemous (e.g. “bank”, “bass”, “run” have several unrelated senses).
- Sense boundaries themselves can be fuzzy/overlapping even for human annotators (inter-annotator agreement is imperfect).
- Requires combining lexical, syntactic, semantic, and world-knowledge cues.

9.3 Categories of WSD Approaches

9.3.1 Knowledge-based Methods

- Lesk algorithm and its variants (see Section 3) — use dictionary glosses/overlap.
- Semantic similarity / graph-based methods over WordNet (e.g. using path length between synsets, or measures like Wu-Palmer, Resnik similarity).
- Selectional restrictions/preferences: use the fact that verbs impose semantic constraints on their arguments (e.g. “eat” selects for an edible object) to rule out incompatible senses.

9.3.2 Supervised Methods

Treat WSD as a classification problem: for each ambiguous word, train a classifier on a sense-tagged corpus (e.g. SemCor, built from WordNet-annotated text).

- Features: surrounding words (bag-of-words context), collocations, POS tags of neighboring words, syntactic relations.
- Classifiers commonly used: Naive Bayes, Decision Lists, Support Vector Machines, and (in modern systems) neural classifiers using contextual embeddings.

- Naive Bayes formulation: choose sense $s^* = \operatorname{argmax}_s P(s) \times \prod P(\text{feature}_j | s)$, using MLE-estimated class-conditional feature probabilities.
- Drawback: requires large amounts of hand-sense-tagged training data (“knowledge acquisition bottleneck”), and a separate classifier is often needed per ambiguous word.

9.3.3 Semi-supervised Methods

- Bootstrapping (Yarowsky Algorithm): start from a small set of seed rules/labeled instances for the highly reliable “one sense per collocation” and “one sense per discourse” heuristics, iteratively label more data, retrain, and repeat, expanding the labeled set each round.
- Reduces dependence on large amounts of manually sense-tagged data compared to fully supervised methods.

9.3.4 Unsupervised Methods (Word Sense Induction)

- Do not rely on a predefined sense inventory; instead cluster the different contexts in which a word occurs (using distributional/context vectors), with each resulting cluster corresponding to an induced “sense”.
- Useful when no sense-tagged resource exists for a language/domain.

9.4 Evaluation

- Metrics: accuracy, precision, recall, F1 against a gold-standard sense-tagged test set.
- Baselines: Most Frequent Sense (MFS) baseline — always pick the most common sense of the word in the training data / WordNet’s first sense — is a surprisingly strong baseline that all WSD systems are compared against.
- Standard shared-task evaluation frameworks: Senseval / SemEval WSD tasks.

9.5 Applications

- Machine Translation (choosing the correct target-language word for an ambiguous source word).
- Information Retrieval (matching query intent to the correct sense of query terms).
- Question Answering, text-to-speech (words with different senses can have different pronunciations), and general semantic understanding pipelines.

End of revision notes.